

Online Appendices for *dSABRE: A SABRE-Style Router for Multi-Core Distributed Quantum Processors*

Sanjiang Li

Centre for Quantum Software and Information, University of Technology Sydney, Australia

Email: sanjiang.li@uts.edu.au

This document collects the online appendices referenced from the main paper. Cross-references with prefix `sec:` and `tab:` (e.g. Section III-C of the main paper) refer to the main manuscript and are not resolved here; section and table numbering in this document is self-contained.

Scope. These appendices provide material needed to verify the algorithmic claims and interpret the comparisons in the main paper. They retain: (i) executable routing semantics, (ii) the completion argument, (iii) the complexity argument and its implementation validation, and (iv) the controlled cross-model and architecture audits that support claims made in Sections III–V. Exploratory experiments, superseded configurations, and implementation-debugging history are intentionally omitted. Complete per-seed JSON records, commands, and patched comparison-tool environments are released with the artefact repository.

APPENDIX A

COMPLETION UNDER FINITE CAPACITY

Theorem 1 is proved in full in Section III-F. This appendix records the implementation properties that proof takes for granted, the recovery cost it leaves out, and how often the mechanism actually fires.

Let f_c be the number of free physical slots in core c and $F_{\text{free}} = \sum_c f_c = P - n$. Both local SWAPs and teleports conserve F_{free} . The router checks before routing that

$$F_{\text{free}} \geq 2K + 1, \quad (\text{A1})$$

and that every core and the core graph are connected, each core has at least four sites, and checkpoint–rollback is enabled. An ordinary teleport is legal only if its landing core has $f_{c_{\text{next}}} \geq \phi_1 = 2$ before the move. It therefore leaves that core non-full; local SWAPs do not alter any core occupancy. Recovery additionally requires the per-core reserve

$$f_c \geq 2 \quad \text{for every core } c, \quad (\text{A2})$$

which ordinary routing does not maintain: measured over every teleport of the 64-qubit suite, some core sits below it at 24.6% of action boundaries (11 177 of 45 527), reaching 46.4% on AE—and at none of them does any core reach zero, which is the floor actually enforced.

The properties the proof assumes are: local SWAPs and teleport macro-actions are atomic, a failure restoring layout,

inverse layout, remaining DAG, counters and trace to the entry state; an ordinary teleport is generated only when its landing core has $\phi_1 = 2$ free sites, and its implementation uses no auxiliary teleport that changes a third core’s occupancy; and the initial layout obeys Eq. (A2), which Section III-G enforces by reserving sites per core. That last one is not cosmetic: the global condition Eq. (A1) alone does not make an arbitrary initial placement recoverable, since every free site could lie outside one completely full core, whose occupied source port would then not be evacuable.

What the capacity ablation switches off: The two rows of Table IV that drop the floor—*no legality floor* and *neither floor nor score*—run the router in score-only mode; *strict floor* raises it to $\phi_1 = 3$ with safe mode still on. Checkpoint–rollback is unaffected—both designs carry it, and neither takes it—but the guaranteed transaction is not reached, and an ordinary teleport reverts to requiring one free slot in its landing core before the move instead of $\phi_1 = 2$. A landing core may therefore end full—exactly what Lemma 1 rules out under the shipped rule—so an occupied port there can no longer be evacuated, the greedy backup those arms fall back on can fail, and that is how the *neither* arm loses two circuits. The relays that backup issues still ask for two free slots at each receiving core, so Eq. (14) is respected by every teleport a relay emits even in the ablated arms; it is ordinary routing, not recovery, that the ablation frees.

Recovery cost: Restoring the reserve moves at most $O(K)$ vacancies across paths of length at most D_K , for $O(KD_K)$ teleports. Co-locating the operands and restoring the occupancy vector adds $O(D_K)$ teleports. A recovery transaction therefore uses $O(KD_K)$ EPR pairs in the worst case. This is a feasibility bound, not an EPR-optimality claim.

Three further facts follow from the same construction. When the meeting core already has headroom the transaction costs exactly d teleports for a gate at core distance d —the information-theoretic lower bound, since each teleport advances one qubit by one core—and when it does not, the relay adds at most d more. The $O(KD_K)$ bound is $\Theta(K^2)$ in the worst case, on a core path whose slack all sits at one end. And charging each gate its stall window, its own transaction and one progressing iteration bounds the whole loop at $|C|(L_{\text{deadlock}} + 2)$ iterations.

TABLE A1

THE SHIPPED DEFAULT—dsABRE WITH THE CAPACITY CONDITIONS, WHOSE COLUMN REPRODUCES TABLE III’S 64-QUBIT EPR FIGURES—AGAINST THE SCORE-ONLY DESIGN IT REPLACED, OVER THREE INITIAL LAYOUTS $\times$ FORWARD-BACKWARD-FORWARD. EPR PAIRS ARE THE REPORTED ROUTE’S; LOWER IS BETTER. THE MECHANISM COUNTERS BELOW ARE TOTALS OVER ALL NINE ROUTES PER CIRCUIT, ON THE *search* CONVENTION OF TABLE V, SO THEY EXCEED WHAT ANY SINGLE REPORTED ROUTE SHOWS. A DASH MARKS A MECHANISM THE DESIGN DOES NOT HAVE; A ZERO MARKS ONE IT HAS AND NEVER USES, AND THE DISTINCTION MATTERS FOR ROLLBACK, WHICH BOTH DESIGNS CARRY AND NEITHER EVER TAKES.

Circuit	Score-only	Default	Δ
ae	200	165	−17.5%
ghz	6	6	0.0%
graphstate	15	15	0.0%
qft	176	180	+2.3%
qnn	495	539	+8.9%
random	718	732	+1.9%
qpeexact	184	187	+1.6%
qaoa	538	555	+3.2%
multiplier	1285	1285	0.0%
Total	3617	3664	+1.3%
Geo. mean	—	—	+0.2%
<i>Mechanism counters, whole suite, all 81 routes</i>			
Recovery invocations	111	42	
guaranteed transactions	—	42	
of which failed	—	0	
Transaction rollbacks taken	0	0	
Reactive room-making calls	104	0	
Routing iterations	207 650	201 776	−2.8%

APPENDIX B

COMPLEXITY: DERIVATION AND VALIDATION

Section III-H states the bound of Eq. (15) and the reasons it is loose. This appendix derives every term of it, records the four structural properties the derivation relies on but does not argue, and reports the measurements that check the derivation against the implementation which produced every result in the paper.

A. Assumptions

Throughout, N is the number of gates, n the number of logical qubits, K the number of cores, M the largest number of physical sites in one core, and D_K and D_M the core-graph diameter and the largest intra-core diameter. Four quantities are treated as constants: the extended-set limit L , the intra-core degree, the core-graph degree, and the number of links joining one adjacent pair of cores. A core therefore has $O(M)$ edges, the core graph has $O(K)$, and the device has $O(K)$ communication ports; the last two are what bound the teleport candidate count at $O(n)$ below.

Four structural properties are used without being re-argued here: (1) the circuit DAG has bounded degree; (2) the front layer is a matching, so a candidate SWAP changes at most two front-layer distance terms; (3) BFS layers of the extended set are matchings, which is what bounds the per-update cost of building E at $O(n + L)$ after a gate retirement and $O(L)$

for its per-core restrictions; and (4) gates retire only from the front layer, so remaining in-degrees and rollback logs can be maintained incrementally rather than rebuilt. Property (4) is the one that would fail silently if it were violated, which is why it is measured in Table A2 alongside (2).

B. Precomputation

All-pairs breadth-first search inside every core costs $O(KM^2)$ for d_{intra} , and $O(KM^2D_M)$ to materialise the intra-core paths, whose KM^2 entries have $O(D_M)$ sites each; the same search on the core graph costs $O(K^2)$ for d_{core} and $O(K^2D_K)$ for the core paths. Dijkstra over the port graph gives the port-level table d_{port} in $O(K^2 \log K)$, and d_{phys} is then composed on demand by Eq. (1) rather than materialised, which avoids the $\Theta(P^2)$ table a whole-chip all-pairs search would build. The precomputation is therefore

$$T_{\text{pre}} = O(KM^2D_M + K^2(D_K + \log K)), \quad (\text{A3})$$

in $O(KM^2D_M + K^2D_K)$ space, which the materialised paths dominate. Storing next-hop or predecessor pointers instead of path sequences would drop the D_M and D_K factors from both time and space, leaving $O(KM^2 + K^2 \log K)$ time and $O(KM^2 + K^2)$ space, and charging $O(D_M)$ or $O(D_K)$ only for each path actually reconstructed. Precomputation is paid once per architecture, not once per circuit, and is excluded from the bounds below and from the compile times reported in Section IV-C.

C. One routing iteration

The extended set is rebuilt only after a gate retires, at the $O(L)$ per update of Section III-E, and hence $O(NL) = O(N)$ over a route.

In one local-routing iteration, each active core enumerates $O(M)$ candidate edges—its front gates are qubit-disjoint by property (2), so $|F_c| \leq M/2$ —and scores each in $O(L)$, since a SWAP moves at most two front-layer operands. Over all active cores the iteration costs $O(n)$, because $\sum_c |F_c| \leq n/2$.

One teleport-selection iteration generates $O(n)$ candidates: front-layer gates are qubit-disjoint, so there are $O(n)$ inter-core source operands, and bounded core degree and link multiplicity leave $O(1)$ adjacent-link choices per operand. If free-slot counts and locations are recomputed from the inverse layout, scanning all K cores costs $O(KM)$. In the current implementation each candidate may additionally scan $O(M)$ sites in its landing core to identify an eviction target or a free site, giving $O(nM)$ over all candidates. Hence

$$T_{\text{tele}} = O(KM + nM) = O(M(K + n)), \quad (\text{A4})$$

which dominates the $O(n)$ local-routing cost. Both scans are implementation artefacts: per-core free-slot counters and free-site sets maintained incrementally would reduce or remove them, leaving $O(n)$ per iteration.

D. Ordinary iterations and recovery

Recovery is invoked after at most $L_{\text{deadlock}} = 4D_K(D_M + 1)$ stalled iterations and, by Theorem 1, retires at least one gate. There are at most N intervals between successive gate retirements, so the number of ordinary iterations is $O(ND_K(D_M + 1))$ and

$$T_{\text{ord}} = O(ND_K(D_M + 1)M(K + n)). \quad (\text{A5})$$

At the beginning of recovery, Lemma 1 gives $f_c \geq 1$ for every core, so restoring the two-site reserve relays at most one vacancy to each of at most K deficient cores. For each relay the current implementation spends $O(KM)$ searching for a donor and at most $O(MD_K)$ moving the vacancy over a core path of length at most D_K , with an $O(M)$ scan at each hop. Reserve restoration therefore costs

$$O(K(KM + MD_K)) = O(KM(K + D_K)).$$

Moving the blocked gate’s operand to the meeting core costs $O(MD_K)$ more and the final intra-core SWAPs $O(D_M) \subseteq O(M)$, both absorbed by that bound. Since each transaction retires at least one gate, there are at most N of them, giving

$$T_{\text{rec}} = O(NKM(K + D_K)). \quad (\text{A6})$$

This is conservative for the scan-based implementation; maintained donor and free-site structures would reduce the search factors, as would recording that recovery fires far less than once per gate (Table V of the main paper).

E. Common bound

Summing extended-set maintenance, Eq. (A5), and Eq. (A6), and using $D_M + 1 = O(D_M)$, $K + n = O(n)$ and $K + D_K = O(K)$ —valid since $D_M \geq 1$, $K = O(n)$ and $D_K \leq K - 1$ —gives the common bound of the main paper,

$$T_{\text{total}} = O(NM(D_K D_M n + K^2)).$$

The K^2 term is the recovery contribution and the M factor the candidate scans, so the two sources of looseness are separable: with maintained free-slot structures and recovery charged at its measured rate rather than once per gate, the same derivation gives $O(ND_K D_M n)$. For a core graph and cores that are bounded-aspect-ratio two-dimensional grids, $D_K = O(\sqrt{K})$ and $D_M = O(\sqrt{M})$, so $D_K D_M = O(\sqrt{P})$ with $P = KM$ and

$$T_{\text{total}} = O(NM(\sqrt{P}n + K^2)).$$

F. Validation

Both scaling rows are ordinary least squares on $\log(\text{time})$ against $\log(\text{CX count})$, over dSABRE’s per-seed compile time with its default parameters; the fitted exponents are 0.97 ($R^2 = 0.95$) on the nine 64-qubit circuits and 2.17 ($R^2 = 0.99$) on the six QFT/QPEexact scalability instances, and the fixed-architecture exponent is 0.92 ($R^2 = 1.00$) and 1.21 ($R^2 = 0.98$) on the 36- and 25-qubit suites. The two must be reported separately: within one suite only the circuit grows, whereas the scalability series grows the device with it

TABLE A2
IMPLEMENTATION CHECKS SUPPORTING THE COMPLEXITY ARGUMENT.

Claim	Measurement	Result
Scaling at fixed architecture	fit versus CX count, 64q suite (9)	$N^{0.97}$
Scaling with the machine	fit versus CX count, scalability (6)	$N^{2.2}$
Front-layer matching	violations across four suites	0
Parallel local routing is rare	active cores per local iteration	1.00–1.01
Shared E versus submitted sweep	total compile time, 39 instances	1745 s $\rightarrow$ 239 s
Shared E versus submitted sweep	geometric-mean EPR	–2.4%
64q shared E versus sweep	compile-time improvement	9.7×
Hierarchical d_{phys}	disagreeing pairs, seven architectures	0 of 385,336

(six, twelve and twenty 5×5 cores), which Eq. (15) charges through K^2 . An earlier draft quoted a single $N^{1.05}$ fit pooled over all 27 circuits; that figure mixed the two regimes and, because the suite times were protocol totals over three seeds while the scalability times were single routes, two timing conventions as well. The shared- E comparison in Table A2 is the relevant validation of the implementation actually used for every reported result. Earlier per-core “taint” sweeps, profiling detail, and alternative lookahead constructions are omitted because they are not part of the algorithm evaluated in the main paper.

Timing protocol against the submitted version: The submitted manuscript reported 801 s for a 360-qubit QFT and this one reports 48 s, and the two are not measurements of the same thing. The architecture differs: the submitted figure was taken on an H-grid of six 9×9 cores (486 physical qubits), while the scalability series specified in Section IV-C holds core size at 5×5 and grows the core count, so 360 qubits is twenty cores and 500 physical sites. Two implementation changes account for the rest: the incremental bookkeeping of 2026-08-07, which is output-identical to what it replaced and was verified as such over 207 routing passes, and the replacement of the per-core extended-set sweep by one shared construction, which is $43 \times$ faster at 360 qubits. The timing boundary is unchanged in both versions: the reported number covers layout generation and every route of the SabreLayout $\times$ forward/backward/forward protocol, on the same machine class and the same three seeds.

APPENDIX C SUPPLEMENTARY EMPIRICAL AUDITS

A. Parameter robustness

On the 64-qubit suite, every default is the minimum in its one-at-a-time sweep: $w_e = 0.25$, $|E| = 20$, $\gamma = 0.9$, and $c_{\text{pen}} = 15$. Removing lookahead ($w_e = 0$) costs 41.4% geometric-mean EPR; setting $|E| = 5$ costs 14.9%, $\gamma = 0.5$ costs 20.7%, and $c_{\text{pen}} = 0$ costs 9.4%. At 25 qubits the curves are largely flat except for removing lookahead, which costs 11.4%. These results support the fixed defaults but do not establish a universal parameter setting. Table A3 sweeps the cost model instead of the score: dSABRE’s margin widens monotonically as c_{tele} , the cost of a teleport in local SWAPs, rises from 10 to 100.

TABLE A3

COST-MODEL SENSITIVITY: GEOMETRIC-MEAN COST REDUCTION OF dSABRE OVER TELESABRE AS c_{tele} VARIES, COMBINED COST BEING $c_{\text{tele}} \cdot \text{EPR} + \text{SWAP}$, ONE LOCAL SWAP BEING THE UNIT. NEGATIVE IS BETTER. EACH SUITE IS TAKEN OVER ITS FULL CIRCUIT SET: SIX AT 25Q AND 36Q, AND AT 64Q THE EIGHT OF NINE ON WHICH TELESABRE CONVERGES—THE SAME BASIS AS TABLE III’S -49.1% . THE TELEPORT COST CANCELS OUT OF THE ROUTING DECISION ITSELF (SECTION III-A), SO THIS REWEIGHTS THE EPR/SWAP COUNTS ALREADY IN TABLE III RATHER THAN RE-ROUTING; CONFIRMED BY A DIRECT RE-ROUTE SWEEP AT $c_{\text{tele}} \in \{10, 100\}$, WHICH REPRODUCED IDENTICAL EPR AND SWAP COUNTS.

c_{tele}	25q	36q	64q
10	-48.4	-58.6	-42.8
20	-49.3	-59.9	-45.3
50	-50.1	-61.1	-47.4
100	-50.4	-61.5	-48.2

TABLE A4

ABLATION ON THE 25-QUBIT SUITE (B-GRID 2×2 OF 4×4 , SIX CIRCUITS), THE SPARSELY-LOADED COUNTERPART TO TABLE IV. **FULL** IS TABLE III’S OWN GEOMETRIC MEAN. EVERY ROW COMPLETES EVERY CIRCUIT.

Configuration	gmEPR	Δ (%)	aborts
Full (baseline)	15.3	—	0
<i>Scoring terms</i>			
No capacity score term ($c_{\text{pen}}=0$)	14.9	-2.5	0
No extended-set lookahead ($w_e=0$)	17.1	+11.4	0
Topological extended set (not BFS)	15.3	+0.2	0
<i>Capacity as a mechanism</i>			
No legality floor (score term only)	15.1	-1.1	0
Neither floor nor score term	25.7	+67.7	0
Strict floor ($\phi_1=r+1$)	15.4	+0.4	0

TABLE A5

GEOMETRIC-MEAN EPR COUNTS ON THE 64-QUBIT SUITE FOR NON-GRID CORE NETWORKS. A DASH DENOTES A TELESABRE NON-CONVERGENCE.

Architecture	TelesABRE	dSABRE	ΔEPR
Four-core heavy-hex ring (9 circuits)	177.4	85.2	-52.0%
Four-core heavy-hex star (8 common circuits)	142.4	83.3	-41.5%
Four-core heavy-hex star (all 9, dSABRE only)	—	101.1	—

B. Mechanism ablation at 25 qubits

Table A4 repeats the main paper’s Table IV at 25 qubits. The two capacity mechanisms remain substitutes at this scale, and the capacity score alone is slightly negative (-2.5%), which is why the main-paper row is read together with the legality floor rather than on its own.

C. Topology sensitivity

Table A5 provides the per-suite results underlying Section IV-C. The ring and star use four 27-qubit heavy-hex cores (108 physical qubits), rather than the six 16-site grid cores used for the 64q H-grid. The same circuit set, hyperparameters, and best-of-three-seed protocol are used.

For the 200-qubit QFT core-granularity control, six 7×7 cores require 353 EPR pairs versus 579 for TelesABRE;

twelve 5×5 cores require 766 versus 1232. The respective reductions are 39.0% and 37.8%. This single control indicates that the reported routing advantage is not confined to one core granularity; it is not a general hardware co-design conclusion.

D. TeleSABRE action audit

The direct comparison is with TelesABRE because both account for local SWAPs and use one-EPR teledata moves. The action spaces nevertheless differ. TelesABRE generates a candidate only when the operand is already adjacent to an empty source port and the selected destination port is empty. dSABRE prices staging and port eviction in d_{prep} and enumerates each legal neighbouring landing core. Thus, availability is a scored macro-action cost in dSABRE rather than a condition that removes the action.

Non-convergence criterion. A TelesABRE failure reported in the main text is the tool’s own verdict: after a fixed number of iterations without progress its safety valve restores the last layout that made progress and records a deadlock, and if the stall persists it ends the attempt. Every seed exhausts its ten internal attempts this way, well inside the wall-clock budget.

In a streamed decision trace, TelesABRE issued 348 of 414 communication operations (84.1%) while same-core front-layer work was pending on the 25-qubit suite; the corresponding count was 2368 of 3982 (59.5%) on the 64-qubit suite. This does not show that TelesABRE destroys local work: none of 407 (25q) or 3725 (64q) teleports moved an operand of a same-core front gate. It establishes the narrower claim used in the paper: the routers apply different ordering policies.

On the 64-qubit suite, telegate accounts for 289 of TelesABRE’s 4478 EPR pairs (6.5%) among converged circuits. Its use is concentrated in AE (26.6%) and QAOA (13.8%); it is at most 3.2% for the other six converged circuits. The comparison should therefore be read as a controlled teledata comparison with a small, reported telegate asymmetry, not as a claim that the two action sets are identical.

E. Telegate as a scored macro-action

Section VI states that scoring a single telegate is straightforward and that grouping is the open problem. We implemented the scored action to check that claim rather than assert it. For a front-layer gate whose operands sit in *adjacent* cores, one macro-action evicts both endpoints of a link, stages each operand beside its own endpoint, consumes one EPR pair to cat-entangle one operand onto the far endpoint, executes the gate there, and disentangles. Neither operand changes core and the cat qubit is measured out inside the same action, so both cores end at their entry occupancy: this is what licenses $c_{\text{cap}} = 0$, and it also means the action cannot violate the capacity invariant of Appendix A. Restricting it to adjacent cores keeps one pair live for one gate; a k -hop telegate would need k pairs held simultaneously, which is the lifetime assumption Appendix D-B prices. Candidates enter the same sorted list as teleports and the best-scoring action of either kind is executed. A telegate receives no lookahead credit,

TABLE A6

WHAT ONE TELEDATA EPR PAIR BUYS ON THE 64-QUBIT SUITE (H-GRID 2×3 OF 4×4), FROM dsABRE’S OWN TRACE. **G/PAIR** IS GATES EXECUTED PER TELEPORT-OPENED RESIDENCE; THE LAST THREE COLUMNS PARTITION THOSE RESIDENCES. A SINGLE-GATE TELEGATE ALWAYS SERVES EXACTLY ONE GATE, SO IT CAN ONLY COMPETE ON THE TRANSIT AND ONE-GATE COLUMNS.

Circuit	EPR	g/pair	transit (%)	one (%)	≥ 2 (%)
AE	165	18.39	3.0	0.0	97.0
GHZ	6	0.83	16.7	83.3	0.0
GraphState	15	1.00	0.0	100.0	0.0
QFT	180	17.49	3.3	0.0	96.7
QNN	539	27.43	0.9	0.7	98.3
Random	732	3.97	20.2	9.8	69.9
QPE-exact	187	18.16	5.3	0.5	94.1
QAOA	555	13.26	1.6	0.0	98.4
Multiplier	1285	17.94	3.3	22.3	74.4
Suite	3664	15.75	6.2	10.5	83.4

because it moves no qubit between cores; that asymmetry is the amortisation trade-off, not an omission.

Across seventeen variants of the score (flat bias, an explicit amortisation penalty, a selective rule that offers a telegate only where relocation would gain nothing in E , and weights on each term), none improves on teledata alone. The score as described above costs +93.5% EPR in geometric mean over the nine 64-qubit circuits; the best variant reaches +1.0%, and only at settings where the telegate has almost stopped firing. Table A6 gives the reason, measured from dsABRE’s own operation trace: a *residence* is the interval a logical qubit spends in one core between teleports, so a teleport-opened residence costs exactly one EPR pair and serves however many two-qubit gates that qubit executes there. One pair serves 15.75 gates on this suite and 332 in the worst case, while an ungrouped telegate serves one by construction. Telegate-when-ever-legal costs $6.1 \times$ the baseline’s EPR. The action wins only where amortisation is impossible: on capacity-tight variants of the same chip, GHZ (0.83 gates/pair) improves from 33 to 16 pairs and GraphState (1.00) from 69 to 31 — both 64-gate circuits in which each qubit interacts once, and both cases a grouped telegate would not help either, since there are no compatible groups to form. Every routing reported here was checked by replaying its emitted operation stream against the architecture and confirming that the executed two-qubit gates are a valid topological order of the input circuit.

F. Compile time

Table A7 gives the per-circuit compile times behind Section IV-C. Each column is the cost of the protocol that produced that tool’s reported result, not of a single attempt: TelesABRE needs one seed, being deterministic here—its three seeds return identical EPR counts on all nine circuits—while dsABRE needs three SabreLayout seeds, whose results genuinely differ (AE: 165/196/212), and pytket-dqc five by its own convention.

The TelesABRE and dsABRE columns were measured in isolation on one machine; the pytket-dqc column is the

TABLE A7

COMPILATION TIME IN SECONDS ON THE 64-QUBIT SUITE (H-GRID 2×3 OF 4×4), WITH THE QFT SCALABILITY SERIES BELOW. THE SEED COUNT IN EACH COLUMN IS THAT TOOL’S OWN REPORTING PROTOCOL. THE GEOMETRIC MEAN IS OVER THE EIGHT CIRCUITS ALL THREE COMPILERS COMPLETE—THE SAME BASIS IN EVERY COLUMN—AND EXCLUDES RANDOM. *TELESABRE FAILS TO CONVERGE. [†]PYTKET-DQC FELL BACK TO PARTITIONINGHETEROGENEOUS. [‡]ITS EMBEDDING DISTRIBUTOR RETURNED, BUT ON ONLY TWO OF FIVE SEEDS WITHIN BUDGET.

Circuit	CX	TelesABRE (1 seed)	dsABRE (3 seeds)	pytket-dqc (5 seeds)
GHZ	63	0.04	0.05	0.3
Graphstate	64	2.74	0.14	0.6
AE	1962	1.78	2.53	291.3
QFT	1966	10.95	3.23	490.9
Random* [†]	1627	—	6.06	171.0
QPEexact	2139	1.48	2.03	458.3
QAOA [‡]	3920	52.32	5.29	899.4
QNN [†]	8126	9.87	10.06	1126.9
Multiplier [†]	13040	136.25	23.70	1392.5
gmean (8)		4.66	1.87	106.6
<i>QFT scalability, 5×5 cores, core count $6 \rightarrow 12 \rightarrow 20$</i>				
QFT 100q	3420	3.6	3.4	—
QFT 200q	7220	60.9	13.5	—
QFT 360q*	13300	—	48.5	—

best-of-5 search behind Table VI, taken from a separate run. Timings of identical work vary by up to $\sim 2.7 \times$ with machine load, so the table should be read for orders of magnitude. Where pytket-dqc fell back to PARTITIONINGHETEROGENEOUS, the reported time spans the whole search and so includes the abandoned embedding attempt: that distributor exhausts its budget on QNN and Multiplier and raises on Random. It did return on QAOA, but only two of its five seeds completed inside the per-circuit budget, so that time is set by the budget rather than by convergence.

The lower block is the QFT scalability series, whose architectures grow with the circuit (6, 12 and 20 cores of 5×5); its rows are therefore not comparable with the fixed-architecture suite above. At 360 qubits TelesABRE spends 1014 s over three seeds and returns no routing, while dsABRE’s 48.5 s spans three SabreLayout seeds, none of which now aborts under the capacity invariant of Section III-F. pytket-dqc is not reported on that series: its counts there were measured on an earlier network construction and have not been re-run, and the comparison of Section IV-E is made on the three fixed-architecture suites.

G. Benchmark provenance and per-seed counts

Table A8 records the CX count and depth of every benchmark circuit after transpilation, and Table A9 the per-seed EPR counts behind every best-of-three figure reported in the main paper.

TABLE A8
BENCHMARK CIRCUIT PROPERTIES (MEASUREMENTS/BARRIERS
STRIPPED)

Suite	Circuit	Qubits	CX gates	Depth	CX/qubit
25q	AE	25	558	395	22.3
	QFT	25	580	173	23.2
	QNN	25	1223	259	48.9
	Random	25	1124	589	45.0
	GHZ	25	24	27	1.0
	Graphstate	25	25	19	1.0
36q	BV	36	17	23	0.5
	DJ	36	35	41	1.0
	W-state	36	70	145	1.9
	VQE-SU2	36	105	56	2.9
	QPEexact	36	1019	347	28.3
	QAOA	36	1200	256	33.3
64q	AE	64	1962	1058	30.7
	QFT	64	1966	446	30.7
	QNN	64	8126	650	127.0
	Random	64	1627	403	25.4
	GHZ	64	63	66	1.0
	Graphstate	64	64	25	1.0
	QPEexact	64	2139	626	33.4
	QAOA	64	3920	493	61.2
	Multiplier	64	13040	22469	203.8

APPENDIX D CROSS-MODEL COMPARISONS

A. pytket-dqc capacity audit

pytket-dqc [1] uses static hypergraph partitioning and amortised gate teleportation, while dSABRE uses dynamic teledata and intra-core routing. Their EPR units are therefore comparable only after stating the capacity and entanglement-lifetime assumptions. Table A11 reports the smallest uniform link-qubit capacity found sufficient to materialise each pytket-dqc distribution. A dagger means the requirement provably exceeds the two ports per B-grid core or the two to three ports per H-grid core. Values prefixed by > are lower bounds because the search budget expired.

The table is an executability audit, not a claim that gate teleportation is inferior in general. Under its more permissive default model, pytket-dqc obtains 34.9, 12.2, and 150.2 geometric-mean e-bits on the 25q, 36q, and 64q suites, respectively, versus dSABRE’s 15.3, 10.5, and 173.3 EPR pairs. Section IV-E explains why these figures must not be read as a like-for-like win/loss comparison.

B. Entanglement lifetime

Every e-bit count above assumes an unbounded entanglement lifetime. Table A12 re-scores the same distributions with a counter that caps gates per EPR pair at $D_{\max}$ [2] and reproduces pytket-dqc’s own cost exactly at $D_{\max}=\infty$. The 64-qubit comparison crosses parity between $D_{\max}=\infty$ and 10 and becomes a 59.6% dSABRE advantage at $D_{\max}=3$. dSABRE needs no adjustment: a teledata pair is consumed by the teleport that creates it.

TABLE A9
dSABRE EPR COUNT UNDER EACH SABRELAYOUT SEED, WITH THE BEST—WHICH IS WHAT THE MAIN TABLES REPORT—AND THE MEDIAN. TELESABRE IS A SINGLE COLUMN BECAUSE ITS THREE SEEDS COINCIDE. GEOMETRIC MEANS ARE OVER THE CIRCUITS TELESABRE COMPLETES, MATCHING TABLE III’S BASIS; THE 360-QUBIT SUITE HAS NONE AND SO HAS NO MEAN.

	dSABRE by seed					
Circuit	0	1	2	best	median	TeleSABRE
<i>25 logical qubits, B-grid</i>						
AE	23	23	23	23	23	26
GHZ	1	1	1	1	1	4
Graphstate	2	2	2	2	2	13
QFT	33	33	33	33	33	46
QNN	47	48	51	47	48	54
Random	181	188	197	181	188	271
gmean (6)				15.3	15.5	31.1
<i>36 logical qubits, B-grid</i>						
BV	1	1	2	1	1	6
DJ	3	3	3	3	3	4
QAOA	134	136	139	134	136	268
QPEexact	63	63	62	62	63	190
VQE-SU2	12	9	16	9	12	28
W-state	7	6	6	6	6	13
gmean (6)				10.5	11.1	27.6
<i>64 logical qubits, H-grid</i>						
AE	165	196	212	165	196	353
GHZ	6	6	6	6	6	16
Graphstate	20	26	15	15	20	62
QFT	180	203	243	180	203	312
QNN	659	539	607	539	607	889
Random	753	763	732	732	753	—
QPEexact	220	187	197	187	197	289
QAOA	559	603	555	555	559	956
Multiplier	1285	1529	1566	1285	1529	1601
gmean (8)				144.7	162.6	284.5
<i>100 logical qubits, H-grid 2×3 of 5×5</i>						
QFT	201	264	262	201	262	248
QPEexact	246	317	411	246	317	586
gmean (2)				222.4	288.2	381.2
<i>200 logical qubits, H-grid 3×4 of 5×5</i>						
QFT	913	820	766	766	820	1232
QPEexact	896	1042	903	896	903	—
gmean (1)				766.0	820.0	1232.0
<i>360 logical qubits, H-grid 4×5 of 5×5</i>						
QFT	2229	1890	2096	1890	2096	—
QPEexact	3168	2709	1938	1938	2709	—

C. DMapS comparison

DMapS [3] uses a different primitive: its cross-chip movement is principally a remote SWAP, which exchanges two states and therefore has the cost of two one-way teledata moves under the EPR accounting used here. In the nine-circuit 64-qubit suite, dSABRE uses $1.43\times$ fewer EPR pairs in geometric mean; DMapS is lower only on GHZ and Graphstate, where partition structure is especially favourable. This

TABLE A10

EPR AND LOCAL-SWAP HEAD-TO-HEAD: DMAPS VS. dsABRE. DMAPS IS BEST OF FIVE KAHYPAR SEEDS (ITS OWN PROTOCOL); dsABRE IS BEST OF THREE SABRELAYOUT SEEDS, MATCHING THE MAIN-TEXT HEADLINE PROTOCOL. “RCX” AND “RSWAP” ARE THE CROSS-CHIP CX AND CROSS-CHIP SWAP COUNTS IN DMAPS’S OUTPUT CIRCUIT; DMAPS EPR = RCX + 2 · RSWAP. EPR $\times$ = DMAPS EPR/dsABRE EPR; LSW $\times$ ANALOGOUSLY FOR LOCAL SWAPS. VALUES < 1 MEAN DMAPS USES FEWER EPRS (RESP. LOCAL SWAPS); VALUES > 1 MEAN dsABRE WINS. ROWS MARKED WITH † ARE CIRCUITS THAT ONLY RUN AFTER OUR UPSTREAM DMAPS PATCHES (3)/(4).

Suite	Circuit	CX	dsABRE EPR	dsABRE ISW	rCX	rSWAP	DMaPS EPR	DMaPS ISW	EPR $\times$	ISW $\times$
64q	ghz	63	6	39	3	0	3	17	0.50	0.44
64q	graphstate	64	15	88	4	0	4	24	0.27	0.27
64q	ae [†]	1962	165	1325	116	115	346	1119	2.10	0.84
64q	qft	1966	180	1480	148	122	392	1152	2.18	0.78
64q	qpeexact	2139	187	1576	158	125	408	1136	2.18	0.72
64q	random	1627	732	3603	378	380	1138	3446	1.55	0.96
64q	qaoa	3920	555	3734	456	400	1256	3199	2.26	0.86
64q	qnn	8126	539	5260	405	264	933	3762	1.73	0.72
64q	multiplier	13040	1285	8969	2376	784	3944	8514	3.07	0.95
64q	gmean (9)		173.3	1236			247.6	841	1.43	0.68

TABLE A11

PYTKET-DQC E-BITS AND REQUIRED UNIFORM COMMUNICATION CAPACITY, COMPARED WITH dsABRE EPR.

Circuit	e-bits	dsABRE EPR	ports needed
<i>25q B-grid (2 ports/core)</i>			
AE [†]	48	23	20
GHZ	3	1	1
Graphstate	4	2	1
QFT [†]	59	33	7
QNN [†]	114	47	> 12
Random [†]	468	181	> 7
<i>36q B-grid (2 ports/core)</i>			
BV	3	1	1
DJ	3	3	1
QAOA [†]	150	134	> 13
QPEexact [†]	45	62	7
VQE-SU2	9	9	1
W-state	6	6	1
<i>64q H-grid (2–3 ports/core)</i>			
AE [†]	120	165	> 28
GHZ	7	6	1
Graphstate	6	15	1
QFT [†]	157	180	> 6
QNN [†]	530	539	> 6
Random [†]	895	732	> 4
QPEexact [†]	75	187	> 5
QAOA [†]	462	555	> 10
Multiplier	2999	1285	> 1

TABLE A12

ENTANGLEMENT-LIFETIME SENSITIVITY ON THE 64Q SUITE: PYTKET-DQC E-BITS WHEN ONE EPR PAIR MAY SERVE AT MOST $D_{\max}$ GATES (COUNTER AS DESCRIBED IN THE TEXT). THE DISTRIBUTIONS SCORED ARE THE ONES TABLE VI REPORTS, LOADED FROM DISK RATHER THAN SEARCHED AFRESH, SO THE $D_{\max}=\infty$ COLUMN IS THAT TABLE’S PYTKET-DQC COLUMN AND THE COUNTER REPRODUCES ITS COST EXACTLY, CIRCUIT BY CIRCUIT. dsABRE EPR (TABLE III) IS $D_{\max}$ -INVARIANT. Δ_3 COMPARES dsABRE AGAINST THE $D_{\max}=3$ COLUMN; NEGATIVE FAVOURS dsABRE.

Circuit	dsABRE EPR	pytket-dqc e-bits at $D_{\max}$					Δ_3 (%)
		∞	10	5	3	1	
ae	165	120	219	382	595	1670	−72.3
ghz	6	7	7	7	7	7	−14.3
graphstate	15	6	6	6	6	6	+150.0
qft	180	157	268	432	652	1777	−72.4
qnn	539	530	1446	2527	3891	11232	−86.1
random	732	895	895	939	1034	1806	−29.2
qpeexact	187	75	158	288	451	1282	−58.5
qaoa	555	462	832	1488	2306	6464	−75.9
multiplier	1285	2999	3869	5149	7164	17882	−82.1
gmean Δ (%)		+15.3	−23.8	−46.0	−59.6	−80.7	

- [2] M. Bandini, D. Ferrari, S. Carretta, and M. Amoretti, “Optimized compilation for distributed quantum computing,” 2026, arXiv:2602.24062.
- [3] T. Luo, Y. Zheng, Y. Deng, and X. Fu, “DMaPS: End-to-end qubit mapping and routing for distributed quantum computing architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 45, no. 5, pp. 2095–2108, 2026, code: <https://github.com/RoccoLoter/DMaPS>.

result is reported as a cross-model comparison, not as a direct baseline: the two routers differ in communication primitive, occupancy behavior, and initial partitioning. Table A10 gives the per-circuit head-to-head the main paper points to; the released artefact contains the machine-readable results and exact environment instructions.

REFERENCES

- [1] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, “CQCL/pytket-dqc,” <https://github.com/CQCL/pytket-dqc>, Jun. 2024.